

7. Устройство за работа с плаваща запетая. Структура на FPU - устройството. Формати на данните. Изпълними операции. Флагове. Формат на инструкциите с плаваща запетая.

Периферно устройство, типове интерфейси и стандартни интерфейси. Методи за контрол на периферния обмен.

7.1 Устройство за работа с плаваща запетая

Увод

Семейството аритметични копроцесори на Intel включва 8087, 80287, 80387SX, 80387DX и 80487SX за използване с микропроцесора 80486SX. Микропроцесорите 80486DX и Pentium/ Pentium Pro съдържат свои собствени вградени аритметични копроцесори. Някои от клонираните 80486 микропроцесори (от IBM и Cytix) не съдържат аритметични копроцесори. Наборите от инструкции и програмирането за всички устройства са почти идентични; основната разлика е, че всеки копроцесор е проектиран да работи с различен микропроцесор на Intel. Тази глава предоставя подробности за цялото семейство аритметични копроцесори. Тъй като копроцесорът е част от 80486DX, Pentium и Pentium Pro и тъй като тези микропроцесори са обичайни, много програми сега изискват или поне се възползват от копроцесор. Семейството от копроцесори, които са обозначени като 80X87, могат да умножават, разделят, събират, изваждат, намират квадратен корен, частичен тангенс, арктангенс и логаритми. Типовете данни включват 16-, 32- и 64-битови цели числа със знак; 18-цифрени BCD данни; и 32-, 64- и 80-битови числа с плаваща запетая. Операциите, извършвани от 80X87, обикновено се изпълняват много пъти по-бързо от еквивалентните операции, написани с най-ефективните програми, използващи нормалния набор от инструкции на микропроцесора. С подобрения копроцесор Pentium операциите се изпълняват около 5 пъти по-бързо от тези, извършвани от микропроцесора 80486 с еднаква тактова честота. Pentium често може да изпълнява инструкция на копроцесор и две цели числа едновременно. Копроцесорът Pentium Pro е подобен на копроцесора Pentium. Добавени са няколко нови инструкции.

Формат на данни на FPU.

Тук се представят видовете данни, използвани с всички членове на семейството на аритметичния копроцесор. (Вижте Таблица 13-1 за списък на всички микропроцесори на Intel и техните придружаващи копроцесори.) Тези типове данни включват цели числа със знак, BCD и числа с плаваща запетая. Всеки тип има специфична употреба в дадена система и много системи изискват и трите типа данни. Програмирането на асемблер с копроцесора често се ограничава до модифициране на кодирането, генерирано от език на високо ниво като C/C++. За да се осъществи такава модификация, са необходими набор от инструкции и някои основни концепции за програмиране, които са представени в тази глава.

Цели числа със знак

Целите числа със знак, използвани с копроцесора, са по същество същите като описаните. Когато се използват с аритметичния копроцесор, целите числа със знак са дълги 16-б (дума), 32-б (късо цяло число) или 64-бита (дълго цяло число). Преобразуването от десетичен формат във формат на цяло число със знак се извършва по същия начин, както за целите числа със знак. Положителните числа се съхраняват във формат с най-левия знаков бит = 0, а отрицателните числа са съхранени в допълнителен код с най-ляв знаков бит 1. 32б. цяло число може да има стойности от -32,768 до +32,767, късото цяло число е в диапазона $\pm 2 \times 10^{+9}$, а дългите цели числа са в диапазона $\pm 9 \times 10^{+18}$. Целочислените типове данни се ползват в някои приложения, които използват аритметичния копроцесор. Фигура 13-1 показва трите форми на данните при цяло число със знак.

TABLE 13-1 Micro-processor and Intel coprocessor compatibility

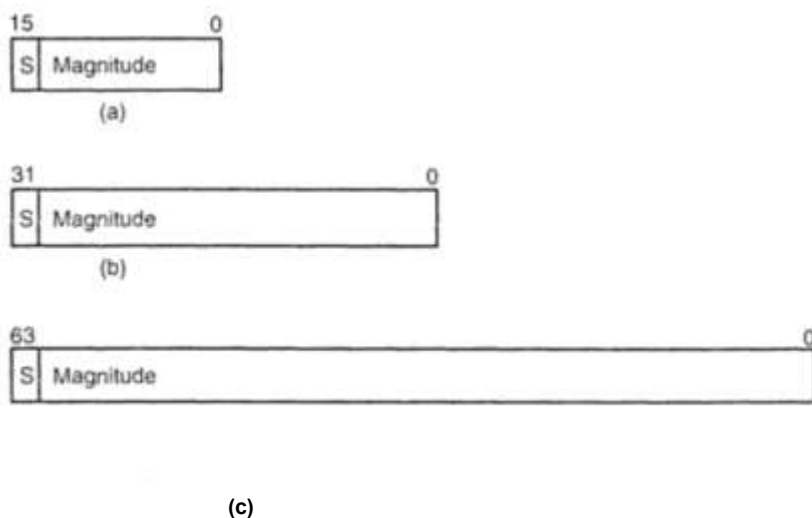
<i>Microprocessor</i>	<i>Coprocessor</i>
8086	8087
8088	8087
80186	80187
80188	80187
80286	80287
80386SX	80387SX
80386DX	80387DX
80486SX	80487SX
80486DX	Built into microprocessor
Pentium	Built into microprocessor
Pentium Pro	Built into microprocessor

EXAMPLE 13-1

0000	0002	DATA1	DW	+2	;16-bit integer
0002	FFDE	DATA2	DW	-34	;16-bit integer
0004	000004D2	DATA3	DD	+1234	;short integer
0008	FFFFFF9C	DATA4	DD	-100	;short integer
000C	00000000000005BA0	DATA5	DQ	+23456	;long integer
0014	FFFFFFFFFFFFFFFF86	DATA6	DQ	-122	;long integer

FIGURE 13-1 Integer formats for the 80X87 family of arithmetic coprocessors:

(a) word, (b) short, and (c) long



Note: S = sign-bit

Двоично кодирани десетични числа (BCD)

Двоично кодираната десетична форма (BCD) изисква 80 бита памет. Всяко число се съхранява като 18-цифрено цяло число в девет байта памет по две цифри на байт. Десетият байт съдържа само знаков бит за 18-цифреното BCD число със знак. Фигура 13-2 показва формата на BCD числото, използвано в аритметичния копроцесор. Както положителните, така и отрицателните числа се съхраняват в прав код, а не в допълнителен код до 10. Директивата DT съхранява BCD данни в паметта, както е показано в Пример 13-2.

EXAMPLE 13-2

0000	DATA1	DT	200	
00000000000000000000	000A	DATA2	DT-10	
80000000000000000010	0014	DATA3	DT10020	

EXAMPLE 13-3

Step	Result
1	100.25 ■ 1100100.01
2	1100100.01 = 1.10010001x 2
3	110 4 01111111 = 10000101
4	Sign = 0 Exponent = 10000101 Significand = 100100010000000000000000

В стъпка трета от Пример 13-3. Изместената степен е експонента, 2^6 или 110, плюс изместване от 01111111 (7FH) или 10000101 (85H). Всички числа с единична точност използват отклонение от 7FH, числата с двойна точност използват изместване от 3FFH, а числата с разширена точност използват изместване от 3FFFH. В Стъпка 4 от Пример 13-3 информацията, намерена в предишни стъпки, се комбинира, за да образува числото с плаваща запетая. Най-левият бит е знаковият бит на числото. В този случай това е 0, защото числото е +100,25]0. Изместената експонента следва знаковия бит. Числото със знак е 23-битово число с подразбиращ се единичен бит. Обърнете внимание, че стойността на числото 1.XXXX е частта XXXX. 1. е подразбиращ се единичен бит, който се съхранява само във формата с разширена точност на числото с плаваща запетая като изрично записан единичен бит. Специални правила важат за няколко числа. Числото 0, например, се съхранява като всички цифри са нули с изключение на знаковия бит, който може да бъде логическа 1 за представяне на отрицателна нула. Плюсът и минус безкрайността се съхранява като логическа 1 в експонентата число със знак на всички нули и знаковия бит, който представлява плюс или минус. NAN (не-число) е невалиден резултат с плаваща запетая, който има само единици в експонентата и е с мантиса, която не е съставена само от нули.

Преобразуване от формат с плаваща запетая.

Преобразуването в десетично число от число с плаваща запетая се обобщава в следните стъпки:

1. Отделете знаковия бит, изместената експонента и мантисата
2. Преобразувайте изместената експонента в истинската експонента, като извадите отместването.
3. Запишете числото като нормализирано двоично число.
4. Преобразувайте го в ненормализирано двоично число.
5. Преобразувайте ненормализираното двоично число в десетично.

Тези пет стъпки преобразуват число с плаваща запетая с единична точност в десетично в Пример 13-4. Забележете как знаковият бит 1 прави десетичния резултат отрицателен. Също така подразбиращият се 1-бит се добавя към нормализирания двоичен резултат в стъпка 3

EXAMPLE 13—4

Step	Result
1	Sign = 1 Exponent r 10000011 Significand = 100100100000000000000000
2	100 = 10000011 - 01111111
3	1.1001001 x2 ⁴
4	11001.001
5	-25.125

Съхранение на данни с плаваща запетая в паметта.

Числата с плаваща запетая се съхраняват в асемблера, използвайки директивата DD за единична точност, DQ за двойна точност и DT за разширена точност. Някои примери за

съхранение на данни с плаваща запетая са показани в Пример 13-5. Асемблерът предоставя емулатор за достъп 8087, ако вашата система не съдържа Pentium Pro, Pentium, 80486 или друг микропроцесор с копроцесор. Емулаторът се предлага с всички езици от високо ниво на Microsoft или като Shareware програми като EM87. Достъпът до емулатора се осъществява чрез включване на оператора OPTION EMULATOR непосредствено след оператора MODEL в програмата. Емулаторът не емулира някои от инструкциите на копроцесора. Не използвайте тази опция, ако вашата система съдържа копроцесор. Във всички случаи трябва да включите .8087, .187, .287, .387, .487 или .587, за да позволи генерирането на инструкции за копроцесор.

EXAMPLE 13-5

0000	C377999A	DATA7	DD	-247.6	;define single-precision
0004	40000000	DATA8	DD	2.0	;define single-precision
0008	4B6F4200	DATA9	REAL4	2.45E+5	;define single-precision
000C		DATA10	DQ	100.25	;define double-precision
	4059100000000000				
0014		DATA11	REAL8	0.001235	;define double-precision
	3F543BF~27136A40	001C			
		DATA12	REAL10	33.9876	;define extended-precision
	400487F34D6A161E4F76				

АРХИТЕКТУРАТА 80X87

80X87 е проектиран да работи едновременно с микропроцесора. Микропроцесорите 80486DX, Pentium и Pentium Pro съдържат свои собствени вътрешни и напълно съвместими версии на 80387. С други членове на семейството, копроцесорът е външна интегрална схема, която разпаралелва повечето връзки на микропроцесора. 80X87 изпълнява 68 различни инструкции. Микропроцесорът изпълнява всички нормални инструкции, а 80X87 изпълнява аритметични инструкции на копроцесора. И микропроцесорът, и копроцесорът могат да изпълняват съответните си инструкции едновременно. Числовият или аритметичен копроцесор е микропроцесор със специално предназначение, който е специално проектиран за ефективно изпълнение на аритметични и трансцендентни операции. Микропроцесорът прихваща и изпълнява нормалния набор от инструкции, а копроцесорът прихваща и изпълнява само инструкциите на копроцесора. Инструкциите на копроцесора всъщност са Escapе инструкции (ESC). Тези инструкции се използват от микропроцесора за генериране на адрес на паметта за копроцесора, така че съпроцесорът да може да изпълни инструкцията на копроцесора.

Вътрешна структура на 80X87

Обща схема на вътрешната логическа структура на този процесор е представена на фигура 1.11.1.

Както се вижда от рисунката, процесорът съдържа 6 байтова опашка за команди, буфер за операнди, регистров LIFO стек с дълбочина 8. Дължината на тези регистри е равна на дължината на разрядната мрежа според стандарта IEEE 754, т.е. 80[b]. Този формат се нарича вътрешен. В него се съхраняват всички типове данни, които се въвеждат за обработка в устройството за работа с плаваща запетая. Диапазонът на този формат е огромен - до $\pm 1,8 \cdot 10^{\pm 4932}$. Но само 19 от тези 4932 цифри на значещата част на десетичните числа могат да бъдат представени в 64 битовата мантиса. При извеждане, резултатите се преобразуват в искания за тях тип и формат.



Фиг. 1.11.1 Структура FPU i8087

При извеждане, резултатите се преобразуват в искания за тях тип и формат. В регистъра на състоянието се съдържа 3 битово поле (Тор), което адресира върховия регистър в регистровия стек. В този смисъл полето Тор представлява указател към стека. Стекът е от тип "винаги готов за четене". Всяка операция запис в стека (от тип POP) първоначално инкрементира указателя, след което записва 80 битовото число в новата върхова клетка. След всяка операция четене от стека (от тип PUSH) указателят се декрементира.

Машинните команди на копроцесора могат да адресират клетките в стека явно и неявно, задавайки отместване относно текущия указател. Тази двойна адресация облекчава предаването на параметри при обръщение към подпрограма. При изпреварващото извличане на командите от ОП те се записват в буферните опашки както на основния процесор, така и на копроцесора. На етапа на предварителния анализ, откритите в копроцесора команди, отнасящи се за основния процесор, се игнорират. Само когато процесорът и копроцесорът срещнат команда за разширение ESC, последната се възприема като команда за FPU. Командите за копроцесора са три типа:

- Без обръщение към оперативната памет ;
- С обръщение към ОП с цел четене на операнд ;
- С обръщение към ОП с цел запис на операнд.

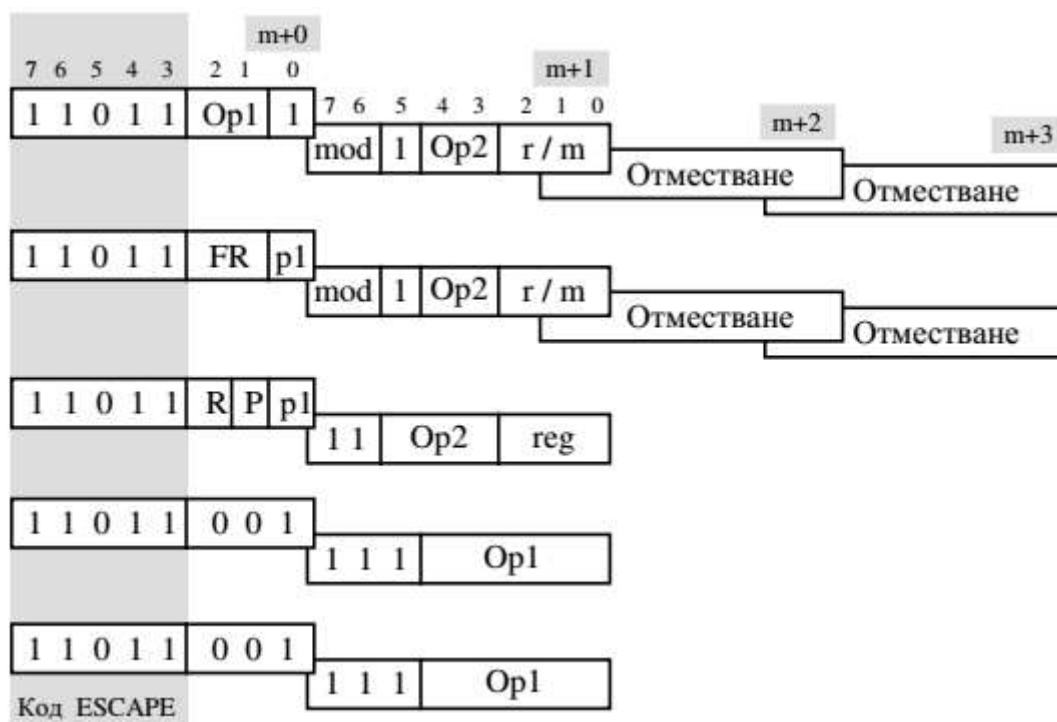
В първия случай, след получаване на командата, копроцесорът пристъпва към нейното изпълнение, а основния процесор продължава извличането и изпълнението на

команди от програмата. Във втория случай, след прочитане на командата основният процесор изпълнява фиктивен цикъл за четене от ОП. Прочетената от паметта дума обаче се приема от копроцесора, когато се окаже върху локалната шина. Ако операндът има по-голяма дължина (32 или 64 бита), по време на фиктивния цикъл копроцесорът приема адреса на операнда и заявява искане за управление на шината. Когато го получи, сам управлява прочитането на останалите порции от операнда. След изчитане на операнда връща управлението на основния процесор и се заема с изпълнение на операцията. В третия случай по време на фиктивния цикъл, който организира основния процесор, копроцесорът прихваща адреса на резултата и го запомня. След като изпълни операция, заявява искане за управление на шината и когато получи правото за това, изпълнява цикли на запис на резултата по запазенения адрес. Тъй като копроцесорът представлява устройство, което е подключено към локалната шина паралелно на основния процесор, е много важно да се организира съвместната им синхронизация. Основната част от тази синхронизация се пада на сигнала BUSY (зает), който копроцесорът формира и подава на вход TEST на основния процесор. В последния цикъл с команда WAIT може да бъде изчакано завършването на операцията в копроцесора. Това изчакване се налага в два случая:

- Когато програмата прави обръщение към копроцесора преди той да е завършил текущата операция ;
- Когато програмата в основния процесор следва да използва резултатът, който копроцесорът изчислява в текущия момент. Синхронизиращата команда WAIT се вмъква в програмния текст автоматично преди всяка команда, използваща копроцесора. Вмъкването е задача на компилатор

Команди на копроцесора

Командите са 69 на брой и имат форматите, представени на фигура 1.12.1.



Фиг. 1.12.1 Формати на командите на копроцесора

Всички команди съдържат в петте старши бита на първия байт кодовата комбинация 11011, с която се отличават от всички останали команди на основния

процесор. Същинската операция за копроцесора се кодира в полетата Op1 (p1) и Op2 (p2). Останалите означения на подполета имат следния смисъл:

- Fr = 00 - кодира къси реални числа (32[b]);

- 01 - кодира къси цели числа (32[b]) ;

- 10 - кодира дълги реални числа (64[b]);

- 11 - кодира цели числа (16[b]) .

- R = 0 - кодира запис на резултата във върховия регистър на стека ; 1 - кодира запис на резултата в друг регистър .

- P = 0 - след операция не модифицира стековия указател ; 1 - след операция модифицира стековия указател .

- reg - съдържа номер (адрес) на регистър в стека . Тъй като адресът на операнда се формира от процесора по време на формалния цикъл за четене от ОП, то полетата *mod* и *r/m* тук имат аналогичен смисъл, с изключение на *mod*=11. В част от командите типът на операндите се задава от полето *Fr* (*Format*), но операциите върху дълги цели като и временни реални числа се изпълняват от отделни команди.

Командите на копроцесора са класифицирани в 6 групи:

1. За прехвърляне на данни. Както между паметта и върховия регистър на стека, така и между регистрите на стека ;

2. За аритметични операции ;

3. За логически проверки ;

4. Изчисляване на стойност на елементарни функции ;

5. Формиране на константни стойности във върховия регистър: 0, 1, π , $\log_{10}2$, $\log_{210}$, $\log_{2e}$, $\log_22$.

6. За управление на копроцесора – зареждане на управляваща дума, зареждане на състоянието, назначаване на режим (инициализация, прекъсване, грешки).

Програмистът има възможност да управлява точността на закръгляне на резултата за всеки от форматите на мантисата (24, 53 или 64 бита). Схемите за закръгляне са 4:

- В положителна посока ($\rightarrow 0 \rightarrow$);

- В отрицателна посока ($\leftarrow 0 \leftarrow$);

- В посока към нулата ($\rightarrow 0 \leftarrow$);

- В противоположна на нулата посока ($\leftarrow 0 \rightarrow$).

Резервирани са представления само на безкрайност (∞) или на знакова безкрайност ($\pm\infty$). Прочитането на “празен” регистър дава операнд NAN. Тази ситуация се използва за откриване на неинициализирани променливи. В копроцесора могат да настъпят 6 особени ситуации, които се отразяват в регистъра на състоянието и могат да бъдат причина за генериране на заявка за прекъсване. Чрез съответните битове в управляващия регистър, всяко прекъсване или комбинация от такива, може да бъде маскирано. Когато особенията ситуация е маскирана, тя се отработва апаратно, като се формира отнапред определен резултат. Особените ситуации са следните:

1. Недействителна операция. Настъпва при препълване или при изпразване на стека, при неопределена ситуация, например при деление 0 на 0, при изваждане на безкрайност, или при използване на NAN като операнд.

2. Препълване. Фиксира се когато резултатът не може да бъде представен в обявения формат. Когато тази ситуация е маскирана, за резултат се приема знакова безкрайност.

3. Антипрепълване. Това е ситуация, при която резултатът е различен от 0, но е много малък по абсолютна стойност и не може да бъде представен в обявения формат. Ако ситуацията е маскирана, резултатът се фиксира с ненормализирана мантиса.

4. Делител =0. Ако ситуацията е маскирана, за резултат се приема знакова безкрайност.

5. Ненормализиран операнд. Ситуацията настъпва, когато прочетеният операнд се окаже с ненормализирана мантиса. Ако ситуацията е маскирана, на този факт не се обръща внимание и операцията се изпълнява.

6. Неточен резултат. Настъпва, когато резултатът следва да бъде закръглен, за да бъде представен в обявения формат. Закръглянето се извършва според текущата схема за закръгляне. Ако ситуацията е маскирана, изчисленията продължават без закръгляне (мантисата на резултата се “отрязва”, т.е. скъсява). Следва да се отбележи, че ако флагът за особена ситуация в регистъра на състоянието е вдигнат, той си остава вдигнат. Флагът може да бъде свален само от специална команда за презапис на флаговете. Презаписът на флаговете дава възможност след изпълнение на дълга последователност от команди (например след изпълнение на подпрограма), да се провери регистъра на състоянието за настъпили при това особени ситуации. В заключение следва да се спомене, че производителността на копроцесора за всяка операция е минимум 100 пъти по-голяма в сравнение с програмните еквиваленти, работещи върху основния процесор.

7.2 Периферни устройства, типове интерфейси и стандартни интерфейси.

Микропроцесорът е идеален в решаването на проблеми, но ако не може да комуникира с външния свят, няма никаква стойност. Тук ще бъдат очертани някои от основните методи за последователна и паралелна комуникация между хора или машини и микропроцесора. I/O интерфейсите обикновено са от паралелен или сериен тип. И двата вида интерфейс имат разнообразни приложения. Например потребителят може да прикачи различни видове нестандартни модули и устройства, използвайки микропроцесорната шина. По-нататък ще покажем как се свързва ADC и DAC модули и DC и стъпкови двигатели към микропроцесора.

Инструкции за I/O обмен

Наборът от инструкции съдържа един тип инструкции, които прехвърлят информация към I/O устройство (OUT) и друг за четене на информация от I/O устройство (IN). Има и инструкции за прехвърляне на низове от данни между паметта и I/O устройство (INS и OUTS – налични във всички IA-32 микропроцесори след 8086/8088).

Таблица 10-1 изброява форматите на всяка инструкция, намираща се в набора от инструкции на микропроцесора.

И инструкциите IN и OUT прехвърлят данни между I/O устройство и акумулатора на микропроцесора (AL, AX или EAX). I/O адресът се съхранява в регистър DX като 16-битов I/O адрес или в байта (p8) непосредствено след операционния код като 8-битов I/O адрес. Intel нарича 8-битовата форма (p8) фиксиран адрес, защото се съхранява с инструкцията, обикновено в ROM. 16-битовият I/O адрес в DX се нарича променлив адрес, защото се съхранява в DX, след което се използва за адресиране на I/O устройството. Други инструкции, които използват DX за адресиране на I/O, са инструкциите INS и OUTS.

Инструкция	Данни[b]	Функция
IN AL,p8	8	Въвежда байт от порт с адрес p8 в AL
IN AX,p8	16	Въвежда дума от порт с адрес p8 в AX
IN EAX, p8	32	Въвежда двойна дума от порт с адрес p8 в EAX
IN AL,DX	8	Въвежда байт от порт с адрес в DX в AL
IN AX,DX	16	Въвежда дума от порт с адрес в DX в AX
IN EAX,DX	32	Въвежда двойна дума от порт с адрес в DX в EAX
INSB	8	Въвежда байтове от порт с адрес в DX в допълнителния даннов сегмент от паметта по адрес, сочен от DI, после DI = DI ± 1

INSW	16	Въвежда дума от порт с адрес в DX в допълнителния даннов сегмент от паметта по адрес, сочен от DI, после DI = DI ± 2
INSD	32	Въвежда дума от порт с адрес в DX в допълнителния даннов сегмент от паметта по адрес, сочен от DI, после DI = DI ± 4
OUT p8,AL	8	Извежда байт от AL в порт с адрес p8
OUT p8,AX	16	Извежда дума от AX в порт с адрес p8
OUT p8,EAX	32	Извежда двойна дума от EAX в порт с адрес p8
OUT DX.AL	8	Извежда байт от AL в порт с адрес в DX
OUT DX.AX	16	Извежда дума от AX в порт с адрес в DX
OUT DX.EAX	32	Извежда двойна дума от EAX в порт с адрес в DX
OUTSB	8	Извежда байт в порт с адрес DX от данновия сегмент от паметта по адрес SI, после SI = SI ± 1
OUTSW	16	Извежда дума в порт с адрес DX от данновия сегмент от паметта по адрес SI, после SI = SI ± 2
OUTSD	32	Извежда двойна дума в порт с адрес DX от данновия сегмент от паметта по адрес SI, после SI = SI ± 4

ТАБЛИЦА 10-1 Инструкции за вход/изход

Всеки път, когато данните се прехвърлят с помощта на инструкцията IN или OUT, I/O адресът, често наричан номер на порт, се появява на адресната шина. Външният I/O интерфейс го декодира по същия начин, по който декодира адрес на паметта. Номерът на фиксирания порт 8-bit (p8) се появява на връзките на адресната шина A7-A0 с битове A15-A8, равни на 00000000,.

Адресите над 2^{16} не са дефинирани за I/O инструкция. 16-битовият номер на променлив порт (DX) активира линиите на адреси A15-A0. Първите 256 адреса на I/O портове (00H-FFH) са достъпни както от инструкции с фиксиран, така и от I/O инструкции с адрес, записан в регистъра DX. I/O адрес от 0100H-FFFFH е достъпен само от инструкции с адрес, записан в регистъра DX. В много специализирани системи за задачи се декодират само най-десните 8 бита от адреса, като по този начин се намалява количеството схеми, необходими за декодиране. В компютъра всички 16 бита на адресната шина могат да указват I/O адрес.

Инструкциите INS и OUTS адресират I/O устройството, използвайки DX регистъра, но не прехвърлят данни между акумулатора и I/O устройството като IN и OUT. Вместо това, тези инструкции прехвърлят данни между паметта и I/O устройството. Адресът на паметта се намира от ES:DI за инструкцията INS и DS:SI за инструкцията OUTS. Както при други стрингови инструкции, съдържанието на указателите се увеличава или намалява, в зависимост от състоянието на флага за посока (DF). Както INS, така и OUTS могат също да бъдат с префикс REP, което позволява повече от един байт или дума да бъдат прехвърлени между I/O и паметта.

Изолирано и общо с паметта адресно пространство за вх./изход I/O

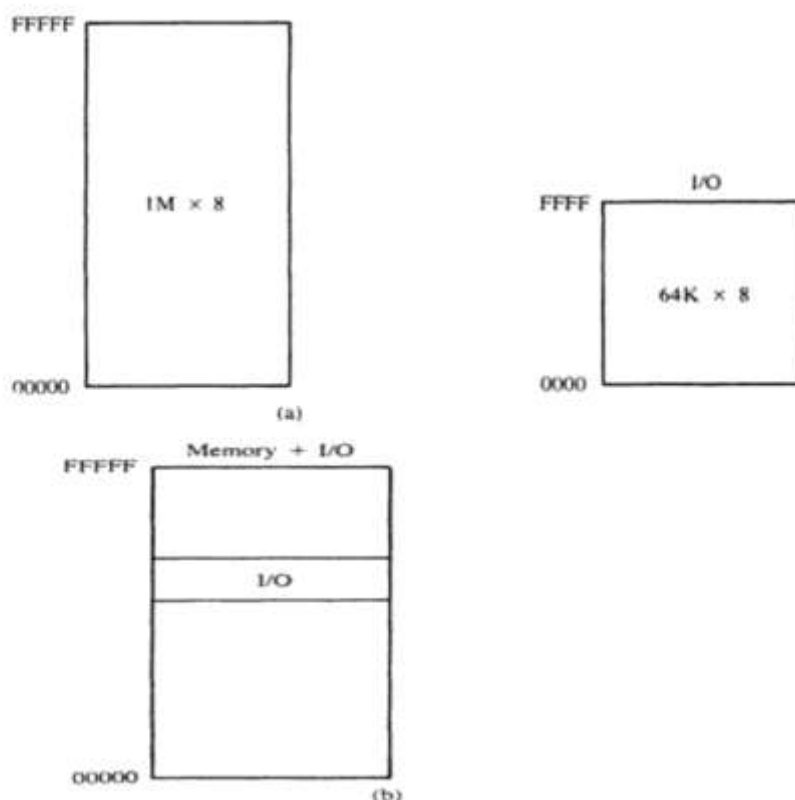
Има два различни метода за свързване на I/O към микропроцесора: изолиран I/O пространство и общо с паметта I/O пространство. В изолираното I/O пространство инструкциите IN, INS OUT и OUTS прехвърлят данни между микропроцесорния акумулатор или паметта и I/O устройството. В общото с паметта I/O.пространство всяка инструкция, която прехвърля данни от/към паметта, може да извърши прехвърлянето

Изолирано I/O пространство.

Микропроцесорните системи, базирана на микропроцесор на Intel, са с изолирано I/O. Терминът изолиран описва I/O адреси, които са изолирани от системната памет в отделно I/O адресно пространство.

Фигура 10-1 илюстрира както изолираните, така и адресните пространства на паметта за всеки микропроцесор Intel 80X86 или Pentium. Адресите за изолирани I/O устройства, наречени портове, са отделни от паметта. В резултат на това потребителят може да разшири и използва паметта до пълния ѝ размер, без да ползва това пространство за I/O устройства. Недостатък на изолирания I/O е, че данните, прехвърлени между I/O и микропроцесора, трябва да бъдат достъпни от отделни инструкции IN, INS, OUT и OUTS. Има обособени отделни управляващи сигнали за I/O пространството (използвайки M/IO и W/R), които показват операция за I/O четене (IORC) или I/O запис (IOWC). Тези сигнали показват, че адресът на I/O порт се появява на адресната шина, която се използва за избор на I/O устройство. В персоналния компютър се използват изолирани I/O портове за управление на периферни устройства. По правило адресът на 8-битов порт се използва за достъп до устройства, разположени на системната платка, като таймер и интерфейс на клавиатурата, докато 16-битов порт се използва за достъп до серийни и паралелни портове, както и видео и дисково устройства и системи.

FIGURE 10-1 The memory and I/O maps for the 8086/8088 microprocessors. (a) Isolated I/O (b) Memory-mapped I/O



Общо с паметта I/O пространство.

За разлика от изолираното I/O пространство, общото с паметта I/O трансфер не използва инструкциите IN, INS, OUT или OUTS. Вместо това той използва всяка инструкция, която прехвърля данни между микропроцесора и паметта. Входно/изходно устройство, разположено в адрес от паметта, се третира като място от паметта на системата.

Основното предимство на картографирания в паметта I/O е, че всяка инструкция за прехвърляне на памет може да се използва за достъп до I/O устройството. Основният недостатък е, че част от системната памет се използва като I/O адреси на портове. Това намалява количеството памет, достъпна за приложенията. Друго предимство е, че IORC и IOWC сигналите нямат функция в I/O система с обща системна и I/O памет могат да намалят количеството схеми, необходими за декодиране.

I/O адреси за вход/изход при персонален компютър

Персоналният компютър използва част от I/O адресно пространство за специални функции. I/O пространството между портове 0000H и 03FFH обикновено е запазено за компютърната система. I/O портовете, разположени на 0400H-FFFFH, обикновено са достъпни за потребителски приложения. Аритметичният копроцесор 80287 използва I/O адрес 00F8H-00FFH за комуникации. Поради тази причина Intel запазва I/O портове 00F0H—00FFH. 80386-Pentium Pro използва I/O портове 800000F8-800000FFH за комуникации към техните копроцесори. Входно-изходните портове, разположени между 0000H и 00FFH, имат достъп чрез инструкциите за вход/изход до порт с фиксиран адрес; портовете, разположени над 00FFH, са достъпни чрез инструкциите за косвена адресация за I/O портове.

Системите от шини (ISA, EISA, VESA и PCI) позволяват I/O портове/регистри и паметта на системата да бъдат свързани към персоналния компютър. ISA шината е 8- или 16-битова и поддържа трансфер на памет или I/O при скорост от 8 MHz.

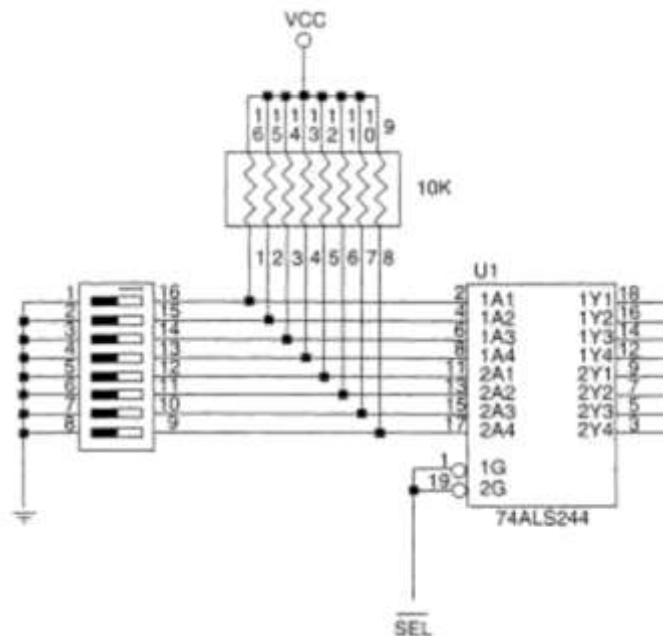
Шината EISA е разширена версия на ISA шината, която поддържа 8-, 16- и 32-битов трансфер между персоналния компютър и паметта или I/O със скорост от 8 MHz. Локалната шина VESA (Video Electronics Standards Association) поддържа 32-битови трансфери между персоналния компютър и I/O или паметта при скорост от 33 MHz. PCI (взаимно свързване на периферни компоненти) поддържа 32- или 64-битов трансфер между персоналния компютър и паметта или I/O със скорост от 33 MHz. Тази шина също така позволява практически всеки микропроцесор да бъде свързан към PCI шината чрез използването на мостов интерфейс. Plug-and-play интерфейс е този, който съдържа памет, която съхранява информация за конфигурацията на системата.

Основни входни и изходни интерфейси

Основното входно устройство има набор от буфери с три състояния. Основното изходно устройство е набор от регистри за данни. Терминът IN се отнася до преместване на данни от I/O устройството в микропроцесора, а терминът OUT се отнася до преместване на данни от микропроцесора към I/O устройството. Буферите с три състояния се използват за изграждане на 8-битовия входен порт, показан на Фигура 10-3. Външните TTL данни (обикновени превключватели в примера) са свързани към входовете на буферите. Изходите на буферите се свързват към шината за данни. Точните връзки на шината за данни зависят от версията на микропроцесора. Например, 8088 има връзки с шина за данни D7-D0, докато 80486 има D31-D0, а Pentium и Pentium Pro имат D63-D0. Схемата от фигура 10-3 позволява на микропроцесора да прочете съдържанието на осемте превключвателя, които се свързват към която и да е 8-битова секция на шината за данни, когато сигналът за избор SEL стане логическа 0.

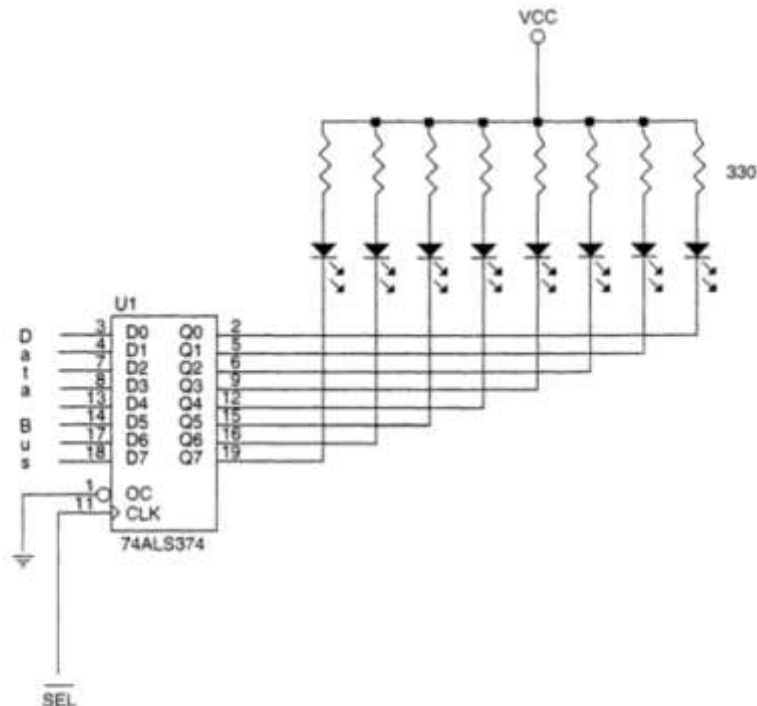
Когато микропроцесорът изпълнява IN инструкция, адресът на I/O порта се декодира, за да се генерира логическата 0 на SEL. Логическа 0, подавана на входовете за управление на изхода (1G и 2G) на буфера 74ALS244, кара входните линии за данни (A) да бъдат свързани към линиите за данни на изхода (Y). Ако се подаде логическа 1 на входовете за управление на изхода на буфера 74ALS244, устройството влиза в режим с висок импеданс, който изключва превключвателите от шината за данни.

FIGURE 10-3 The basic input interface illustrating the connection of eight switches. Note that the 74ALS244 is a three-state that controls the application of the switch data to the data bus.



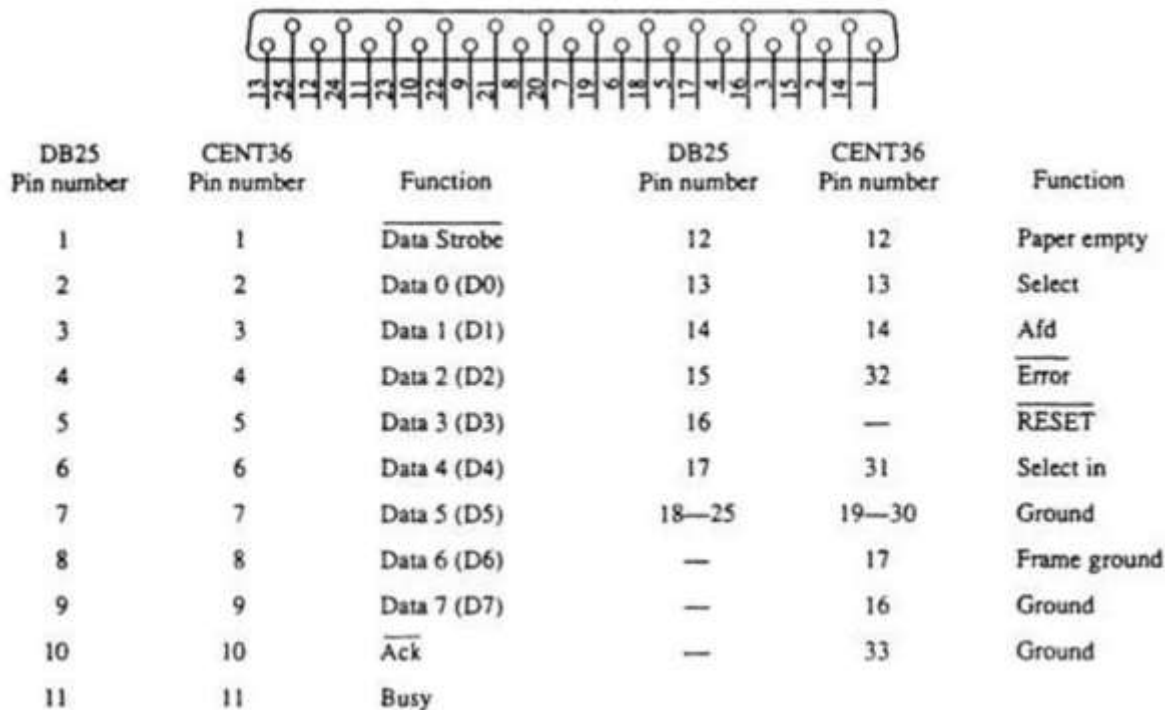
Когато се изпълни инструкция OUT, данните от AL, AX, или EAX се прехвърлят към буферите чрез шината за данни. Тук D входовете на осемте буфера 74ALS374 са свързани към шината за данни за приемане на изходните данни, а Q изходите на буфера са свързани към светодиодите. Когато изходът Q стане логическа 0, светодиодът светва. Всеки път, когато се изпълнява инструкцията OUT, сигналът SEL към буфера се активира, приемайки изходните данни към буферите от 8-битовата шина за данни. Данните се запазват непроменени до изпълнението на следващата OUT инструкция.

FIGURE 10-4 The basic output interface connected to a set of LED displays



Протокол

Много I/O устройства приемат или издават информация много по-бавно в сравнение с микропроцесора. Друг метод за I/O контрол, наречен ръкостискане **handshaking** или **polling** синхронизира I/O устройството с микропроцесора. Примерно устройство, което изисква ръкостискане, е паралелен принтер, който отпечатва 100 знака в секунда (CPS). Очевидно е, че микропроцесорът определено може да изпрати повече от 100 символа/с. към принтера, така че това е начин да се забави микропроцесора, за да съответства на скоростите с които приема принтера.



Фиг. 10-5. DB25 съединител/конектор се ползва от принтерите тип Centronics за извеждане на сигналите на паралелния порт/интерфейс

ASCII данните, които трябва да бъдат отпечатани от принтера, се поставят върху D7-D0 и се подава строб-импулс към STB линия. Строб сигнала изпраща данните към принтера, за да могат да бъдат отпечатани. Веднага щом принтерът получи данните, той поставя ниво логическо 1 върху линия BUSY, което показва, че е зает с отпечатване на данни. Микропроцесорът проверява/тества линията BUSY периодично, за да реши дали принтерът е зает. Ако принтерът е зает, микропроцесорът чака; ако не е зает, микропроцесорът изпраща следващия ASCII символ към принтера. Процесът на разпитване на принтера се нарича ръкостискане или запитване.

Има проста процедура за отпечатване на символи на принтера. Тя тества линията BUSY и изпраща данни към принтера, ако той не е зает. Процедурата PRINT отпечатва ASCII-кодираното съдържание на BL само ако флагът BUSY е логическа 0, което показва, че принтерът не е зает.

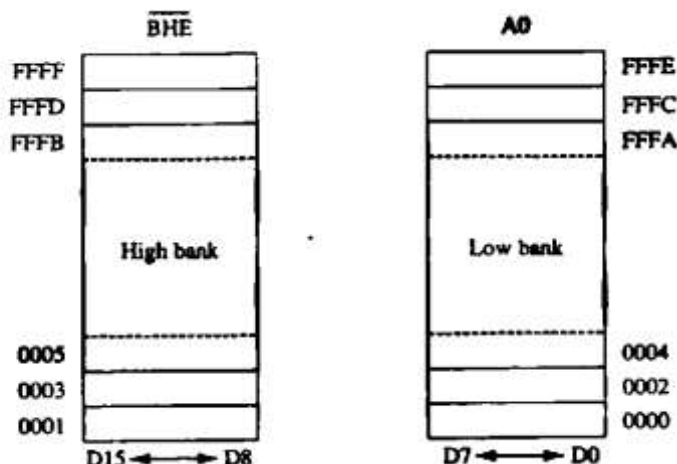
Декодиране на адресите на I/O порт

Декодирането на адрес на I/O порт е много подобно на декодирането на адрес на паметта. Общите с паметта I/O адреси се декодират точно като адреси на паметта. IORC и IOWC не се използват, тъй като няма инструкция IN или OUT. Решението да се ползва обща I/O пространство с паметта често се определя от размера на системата с памет и разположението на I/O устройствата в системата.

Основната разлика между декодирането на паметта и изолираното I/O декодиране е броят на адресните изводи, свързани към декодера. Декодираме A32-A0, A23-A0 или A19-A0 за памет и A15-A0 за адреса в изолирано I/O пространство. Понякога, ако I/O устройствата използват само фиксирано I/O адресиране, ние декодираме само A7-A0. Друга разлика е, че използваме IORC и IOWC за активиране на I/O устройства за операция за четене или запис. В по-ранните версии на микропроцесора IO/M = I и RD или WR се използват за активиране на I/O устройства. В най-новите версии на микропроцесора M/IO = 0 и W/R дъгата се използва за активиране на I/O устройства. Фигура 10-6 илюстрира декодер 74ALS138, който декодира 8-битови I/O портове F0H-F7H. (Предполагаме, че системата ще използва само I/O портове 00H-FFH за този декодер.) Този декодер е идентичен с декодер на адрес на паметта, освен че свързваме само адресни битове A7-A0 към входовете на 8- и 16-битови I/O портове.

Декодирането на адреса на I/O порта вероятно е по-просто от декодирането на адрес на паметта (поради броя на битовете), ще обясним как данните се прехвърлят между микропроцесора и 8- или 16-битовите I/O устройства. Данните, прехвърлени към 8-битово I/O устройство, съществуват в една от I/O банките в 16-битов микропроцесор като 8086, 80286, 80186 или 80386SX. I/O системата съдържа две 8-битови банки памет, точно както паметта. Това е илюстрирано на Фигура 10-9, която показва отделните I/O банки за 16-битова система като 80386SX.

FIGURE 10-9 The I/O banks found in the 8086, 80186, 80286, and 80386SX



32-битовите I/O портове не са често срещани. Някога обещаващата системна шина EISA поддържа 32-битов I/O, както и VESA локална и текуща PCI шина, но те се срещат само в някои компютърни системи.

ПРОГРАМИРУЕМ ПЕРИФЕРЕН ИНТЕРФЕЙС

Програмируемият периферен интерфейс 82C55 (PPI) е много популярен евтин интерфейс компонент, който се намира в много приложения. PPI има 24 извода за I/O, програмируеми в групи от 12 пина, които се използват в три отделни режима на работа. 82C55A може да свързва всяко TTL-съвместимо I/O устройство към микропроцесора. 82C55A (CMOS версия) изисква състояния на изчакване на вмъкване, ако се работи с микропроцесор, използващ тактова честота по-висока от 8 MHz. Освен това осигурява поне 2,5 mA ток на поглъщане (логическа 0) на всеки изход с максимум 4,0 mA. Тъй като I/O устройствата са по своята същност бавни, листите за изчакване, използвани по време на I/O трансфери, не оказват съществено влияние върху скоростта на системата. 82C55 все още намира приложение (съвместим за програмиране, въпреки че може да не се появи в системата като дискретен 82C55) дори в най-новата компютърна система, базирана на 80486 или Pentium Pro. 82C55 се използва за интерфейс към клавиатурата и паралелния порт за принтер в много от тези персонални

компютри. Той също така контролира таймера и чете данни от интерфейса на клавиатурата. Фигура 10-13 илюстрира pin-out диаграмата на 82C55. Неговите три I/O порта (маркирани А, В, и С) са програмирани в групи от 12 крачета. Връзките от група А се състоят от порт А (РА7—РА0) и горната половина на порт С (РС7-РС4), а група В се състои от порт В (РВ7-РВ0) и долната половина на порт С (РС3-РС0). 82C55 се избира от своя CS извод за програмиране и за четене или запис на порт. Изборът на регистър се осъществява чрез входните линии А1 и А0, които избират вътрешен регистър за програмиране или работа. Таблица 10-2 показва назначенията на I/O портове, използвани за програмиране и достъп до I/O портовете. В персоналния компютър 82C55 или негов еквивалент се декодира на I/O адреси 60H-63H.

FIGURE 10-13 The pin-out of the 82C55 peripheral interface adapter (PPI)

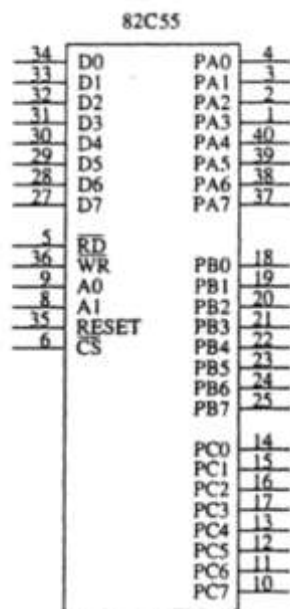


TABLE 10-2 I/O port assignments for the 8255

A_1	A_0	Function
0	0	Port A
0	1	Port B
1	0	Port C
1	1	Command Register

82C55 е просто устройство за взаимодействие с микропроцесора и програмата. За да се чете или записва 82C55, CS входът трябва да бъде логическа 0 и правилният I/O адрес трябва да бъде приложен към изводите А1 и А0. Останалите адресни линии на порта не участват, що се отнася до 82C55, и са външно декодирани, за да се избере 82C55. Фигура 10-14 показва 82C55, свързан към 80386SX, така че да функционира на 8-битови адреси на I/O порт СОН (порт А), С2Н (порт В), С4Н (порт С) и С6Н (команден регистър). Този интерфейс използва долната банка на I/O картата 80386SX. Всички изводи 82C55 са директно свързани към 80386SX, с изключение на извода CS. CS линията се декодира и избира от декодер 74ALS138. Входът RESET към 82C55 инициализира устройството всеки път, когато микропроцесорът се нулира. Вход RESET към 82C55 кара всички портове да бъдат настроени като прости входни портове, като се използва режим 0 на работа. Тъй като изводите на порта са вътрешно програмирани като входни линии, при нулиране повредата се предотвратява, когато захранването се подаде за първи път към системата. След RESET не са необходими други команди за програмиране на 82C55, ако се използва като входно устройство и на

трите порта. 82C55 бе свързан към PC/XT компютър на адреси на портове 60H-63H за управление на клавиатурата, а също и за управление на високоговорителя, таймера и други вътрешни устройства, като например разширяване на паметта.

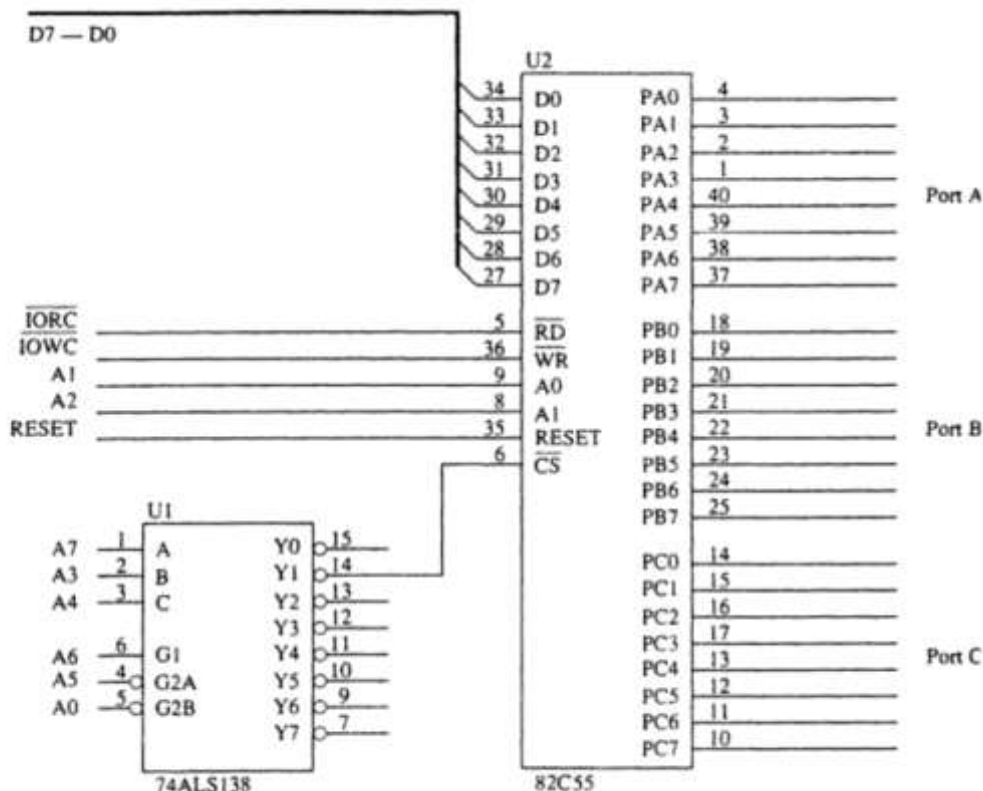


FIGURE 10-14 The 82C55 interfaced to the low bank of the 80386SX microprocessor

Програмиране на 82C55

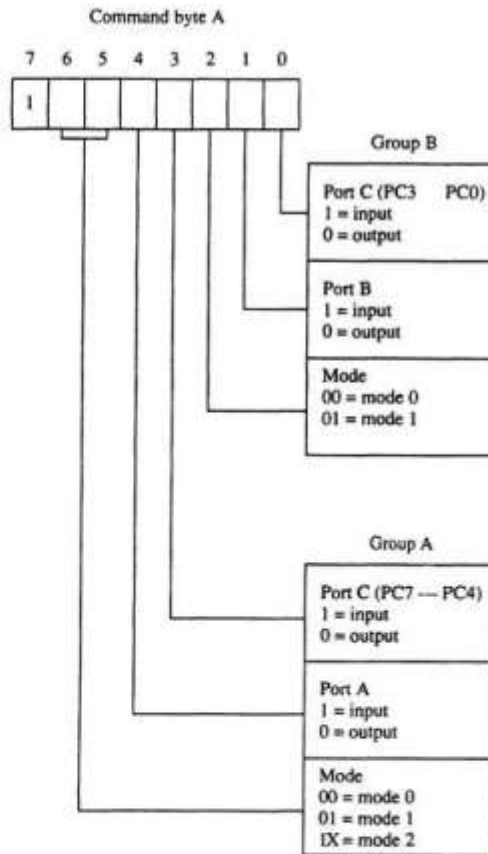
82C55 е лесен за програмиране, тъй като съдържа само два вътрешни командни регистъра, както е илюстрирано на Фигура 10-15. Бит 7 избира или команден байт А, или команден байт В. Команден байт А програмира функцията на група А и В, докато команден байт В задава (1) или нулира (0) битове на порт С само ако 82C55 е програмиран в режим 1 или 2.

Изводите от група В (порт В и долната част на порт С) са програмирани като входни или изходни изводи. Група В може да работи или в режим 0, или в режим 1. Режим 0 е основният режим на вход/изход, който позволява изводите от група В които са програмирани като прости входни и изходни линии.

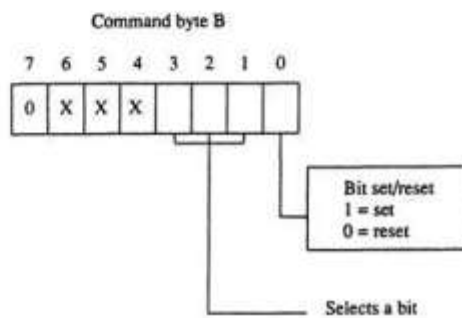
Операция в режим 1 е стробираната операция за връзка с група В, при която данните се прехвърлят през порт В, а сигналите за ръкостискане се предоставят от порт С. Изводите от група А (порт А и горната част на порт С) също са програмирани като входни или изходни изводи. Разликата е, че група А може да работи в режими 0, 1 и 2.

Работата в режим 2 е двупосочен режим на работа за порт А. Ако 0 се запише в бит 7 на командния байт, се избира команден байт В. Тази команда позволява всеки бит от порт С да бъде зададен (1) или нулиран (0), ако 82C55 работи в режим 1 или 2. В други случаи този команден байт не се използва. Често се ползва функцията за настройка/нулиране на битове в системата за управление, за да се установи или нулира контролен бит на порт С.

FIGURE 10-15 The command byte of the command register in the 82C55. (a) Programs ports A, B, and C (b) Sets or resets the bit indicated in the select a bit field



(a)



(b)

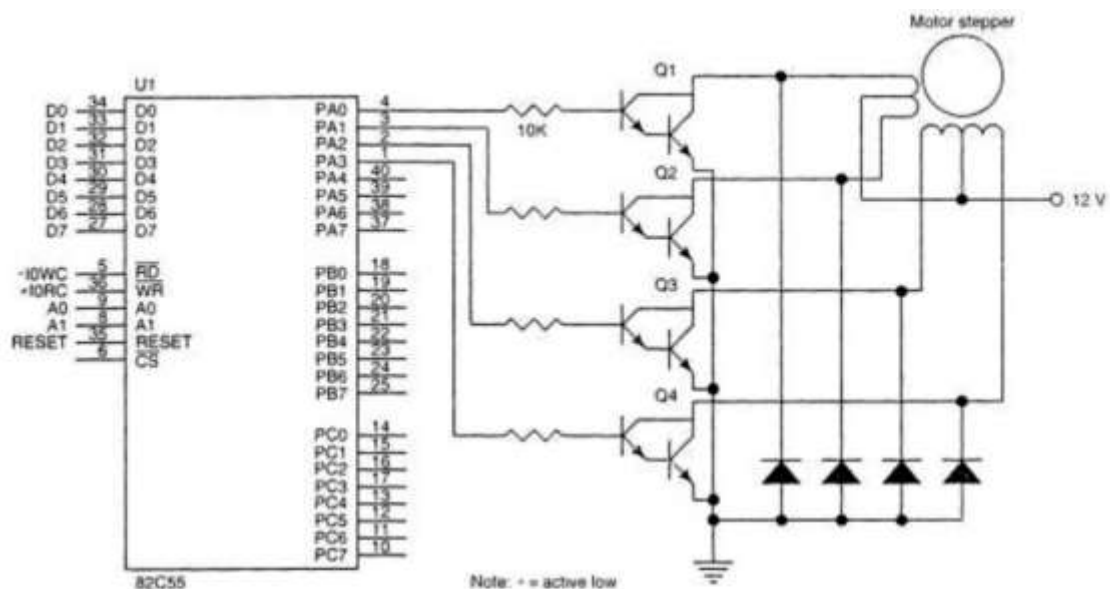


FIGURE 10-18 A stepper motor interfaced to the 82C55. This illustration does not show the decoder.

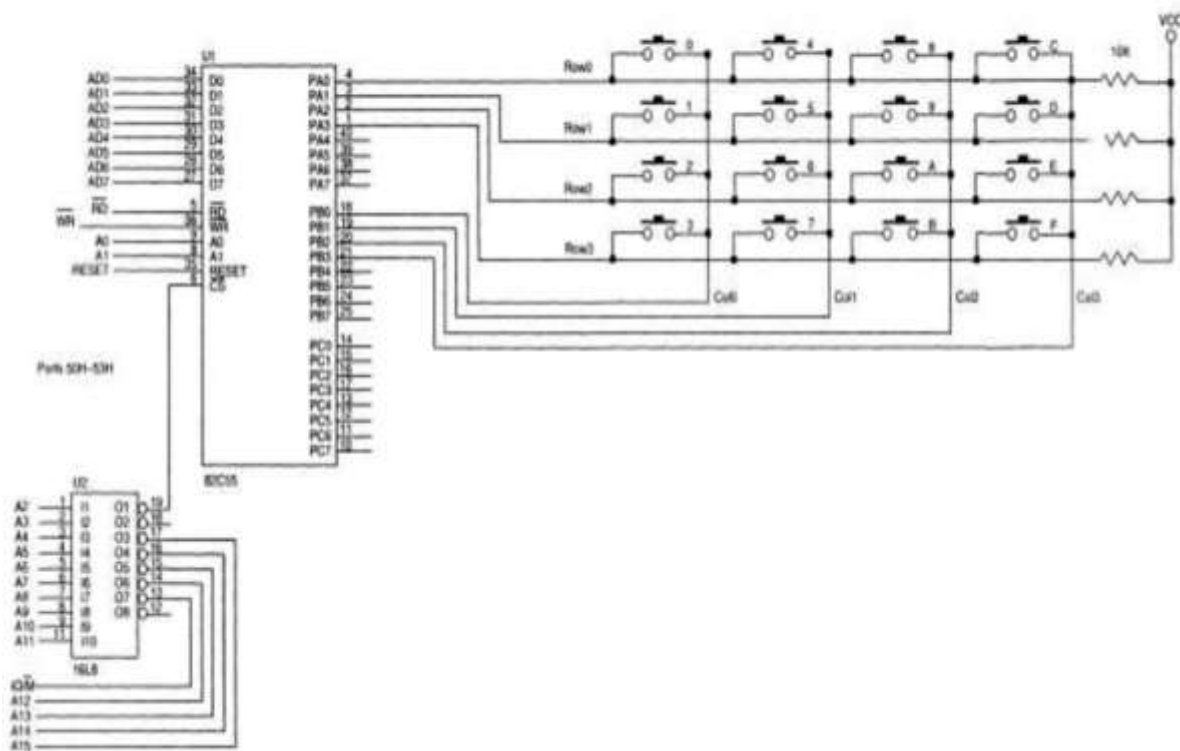
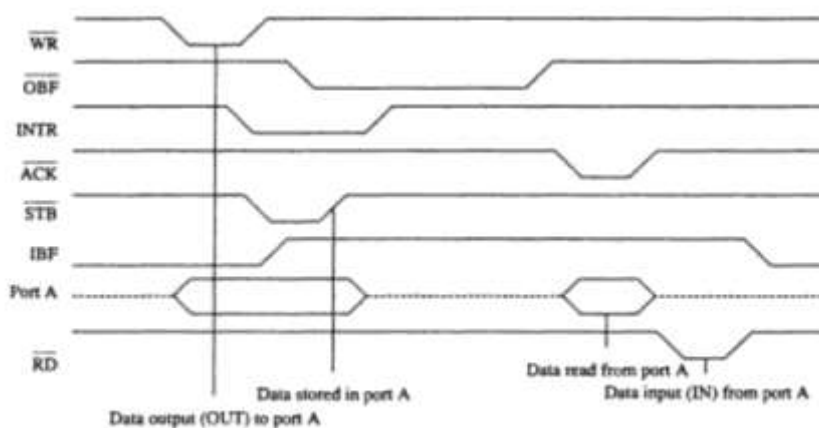


FIGURE 10-19 A 4 x 4 keyboard matrix connected to an 8088 microprocessor through the 82C55 PPIA

Фигура 10-19 илюстрира матрица с малки ключета-бутони, която съдържа 16 превключвателя, свързани към портове А и В на 82C55. В този пример превключвателите са оформени в матрица 4 x 4, но може да се използва всяка матрица, като например 2 x 8. Клавишите са организирани в четири реда (ROW0-ROW3) и четири колони (COL0-COL3). Всеки ред е свързан през резистор 10 K към +5V, за да се осигури в реда високо ниво, когато нито един бутон не е натиснат.

The diagram shows the 82C55 PPI on the left and a Printer on the right. The 82C55 has pins PB₀, PB₇, PC₂, and PC₄. The Printer has pins D₀, D₇, $\overline{ACK}$, and $\overline{DS}$. The connections are as follows:

- PB₀ and PB₇ are connected to D₀ and D₇ respectively, with bidirectional arrows indicating data flow.
- A large arrow labeled "ASCII" points from the 82C55 to the Printer, indicating data transfer.
- PC₂ is connected to $\overline{ACK}$ from the Printer, with an arrow pointing from the Printer to the 82C55.
- PC₄ is connected to $\overline{DS}$ from the Printer, with an arrow pointing from the Printer to the 82C55.

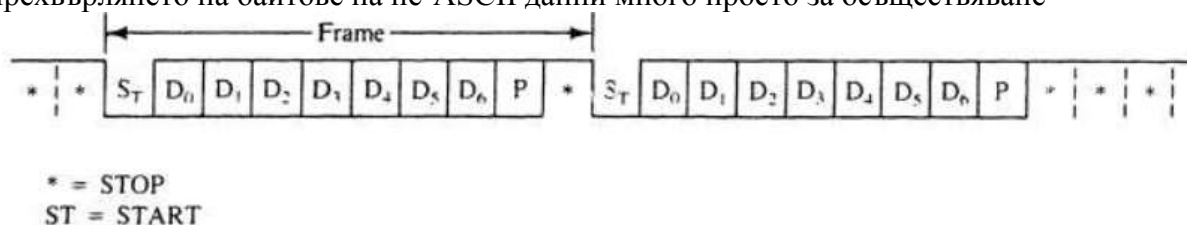


PC16550D на National Semiconductor Corporation е програмируем комуникационен интерфейс, предназначен да свързва почти всеки тип серийен интерфейс. 16550 е универсален асинхронен приемо/предавател (UART), който е напълно съвместим с микропроцесорите на Intel. 16550 работи при скорости 0-1,5 М Baud. Скоростта на предаване е броят на битовете, прехвърлени в секунда, включително начало, стоп, данни и контролен бит. 16550 включва също така програмируем генератор на скорост на предаване и отделни FIFO за входни и изходни данни за облекчаване на натоварването на микропроцесора. Всеки FIFO съдържа 16 байта памет. Това е най-разпространеният комуникационен интерфейс, намиращ се в

модерното оборудване, използвано от микропроцесори, включително персоналният компютър и много модеми.

Асинхронно серийно предаване

Асинхронните серийни данни се предават и приемат без използване на синхронизиращ тактов сигнал. Фигура 10-42 илюстрира два кадъра с асинхронни серийни данни. Всеки кадър съдържа стартов бит, седем бита данни, бит за контрол по четност/нечетност и един стоп бит. На тази фигура кадърът, който съдържа един ASCII знак, има 10 бита. Повечето комуникационни системи за комутируема връзка използват 10-бита за асинхронни серийни данни с еднакъв контрол по четност. Повечето интернет услуги използват 10-бита, но не използват паритет. Вместо това се прехвърлят 8-бита, заменяйки контрола по четност с бит данни. Това прави прехвърлянето на байтове на не-ASCII данни много просто за осъществяване



Фиг. 10-42 Времедиаграма на асинхронно серийно предаване

16550 Функционално описание

Фигура 10-43 илюстрира изводите на 16550 UART. Това устройство се предлага в 40-пинов DIP корпус. Две напълно отделни секции са отговорни за комуникацията на приемника и предавателя. Тъй като всяка от тези секции е независима от другата, 16550 може да функционира в симплексен, полудуплексен или пълен дуплекс режим. 16550 има вградени FIFO (First-in, first-out) памети за приемника и предавателя. Тъй като всеки е с обем 16 байта, UART изисква обслужване от микропроцесора само след получаване на 16 байта данни. Опащката съдържа 16 байта, преди микропроцесорът да изчака предавателя. FIFO прави този UART идеален при свързване към високоскоростни системи, тъй като е необходимо по-малко време за обслужването му. Примерна симплексна система е, когато предавателят или приемникът се използва самостоятелно, в FM (с честотна модулация) радиостанция. Примерна полудуплексна система е гражданска радиостанция, където предаваме и приемаме, но не и двете едновременно. Пълнодуплексната система позволява предаване и приемане в двете посоки едновременно. Примерна система за пълен дуплекс е телефонът. 16550 може да управлява модем (модулятор/демодулятор), който е устройство, което преобразува TTL нивата на серийни данни в аудио тонове, които могат да преминават през телефонната линия и оборудване. Шест крачета на 16550 са предвидени за управление на модем: DSR (готов набор от данни), DTR (готов за предаване данни), CTS (Готов за изпращане), RTS (заявка за изпращане), RI (индикатор за звънене) и DCD (откриване на носеща).

FIGURE 10–43 The pin-out of the 16550 UART



Програмиране на 16550

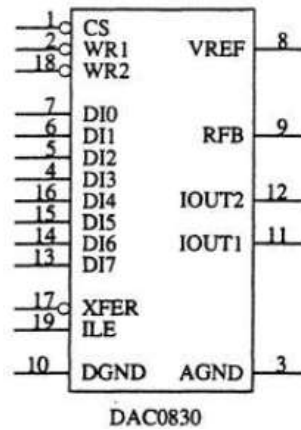
Програмирането на 16550 е сравнително просто. Програмирането е процес от две части, който включва инициализация и оперативен диалог при трансфера.

- **Инициализиране на 16550.** Инициализация се прави след нулиране на хардуера или софтуера, се състои от две части: програмиране на регистъра за управление на линията и генератор на скорост на предаване. Регистърът за управление на линията избира броя на битовите данни, броя на стоп битовите и бита за паритет/контрол (четно/нечетно). Генераторът на скорост на предаване е програмируем делител, който определя скоростта на предаване и приемане.

АНАЛОГОВО-ЦИФРОВ (ADC) И ЦИФРО-АНАЛОГОВ (DAC) ПРЕОБРАЗОВАТЕЛ

Аналогово-цифровите (ADC) и цифрово-аналоговите (DAC) преобразуватели се използват за свързване на микропроцесора с аналоговия свят. Много събития, които се наблюдават и контролират от микропроцесора, са аналогови събития. Те често включват от говор, до управление на двигатели и подобни устройства. За да свържем микропроцесора с тези събития, трябва да разберем интерфейса и управлението на ADC и DAC, които са преобразуватели между аналоговите и цифровите данни. Цифрово-аналоговият преобразувател DAC0830 е често срещан и евтин цифрово-аналогов преобразувател. DAC0830 (продукт на National Semiconductor Corporation) е 8-битов преобразувател, който преобразува 8-битово двоично число в аналогово напрежение. Предлагат се и други преобразуватели, които преобразуват от 10-, 12- или 16-битови цифрови данни в аналогово напрежение.

FIGURE 10-48 The pin-out of the DAC0830 digital-to-analog converter



Броят на нивата на напрежението, генерирани от преобразувателя, е равен на броя на комбинациите на двоичните входове. Следователно, 8-битов преобразувател генерира 256 различни нива на напрежение, 10-битов преобразувател генерира 1024 нива и т.н. DAC0830 е средноскоростен преобразувател, който трансформира цифров вход в аналогов изход за приблизително за 1 μ s. Фигура 10-48 показва корпуса на DAC0830. Това устройство има набор от осем даннови шини за прилагане на цифровия входен код и двойка аналогови изходи отбелязани с Iout1 и Iout2, които са проектирани за свързване с външен операционен усилвател. Тъй като това е 8-битов преобразувател, неговото изходно стъпково напрежение се определя като VRFF (опорно напрежение), разделено на 255.

Свързване на DAC0830 към микропроцесора. DAC0830 се свързван към микропроцесора, както е показано на Фигура 10-50. Тук PAL16L8 се използва за декодиране на DAC0830 на 8-битов I/O портов адрес 20H. Всеки път, когато се изпълнява инструкция OUT 20H.AL, нивото на линиите от шината за данни AD0-AD7 се предава към преобразувателя в DAC0830. Операционният усилвател 741 заедно с -12V ценеровото опорно напрежение прави пълното изходно напрежение да бъде равно на +12V. Изходът на операционния усилвател се подава към драйвер, който захранва 12V DC двигател. Този драйвер е усилвател с транзистор Дарлингтън за мощни двигатели. Този пример показва преобразувателя, задвижващ двигател..

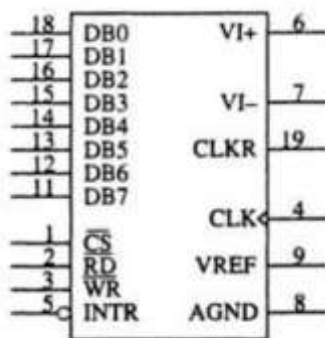
Аналогово-цифров преобразувател ADC0804

Често срещан евтин ADC е ADC0804, който принадлежи към семейство преобразуватели, които са идентични с изключение на точността. Това устройство е съвместимо с широка гама микропроцесори като семейството на Intel. Има и по-бързи АЦП и някои с по-голяма разрешаваща способност от 8-бита. Това устройство е подходящо за много приложения, които не изискват висока степен на точност. ADC0804 изисква до 100 μ s време за преобразуване на аналогово входно напрежение в цифров изходен код.

Фигура 10-51 показва изводите на преобразувателя ADC0804 (продукт на National Semi-conductor Corporation). За да работи с преобразувателя, на линията WR се подава импулс при заземен CS, за да започне процеса на преобразуване. Тъй като този преобразувател изисква значително време за преобразуването, линията INTR сигнализира за края на преобразуването. Вижте Фигура 10-52 за времева диаграма,

която показва взаимодействието на управляващите сигнали. Както се вижда, започва

FIGURE 10-51 The pin-out of the ADC0804 analog-to-digital converter



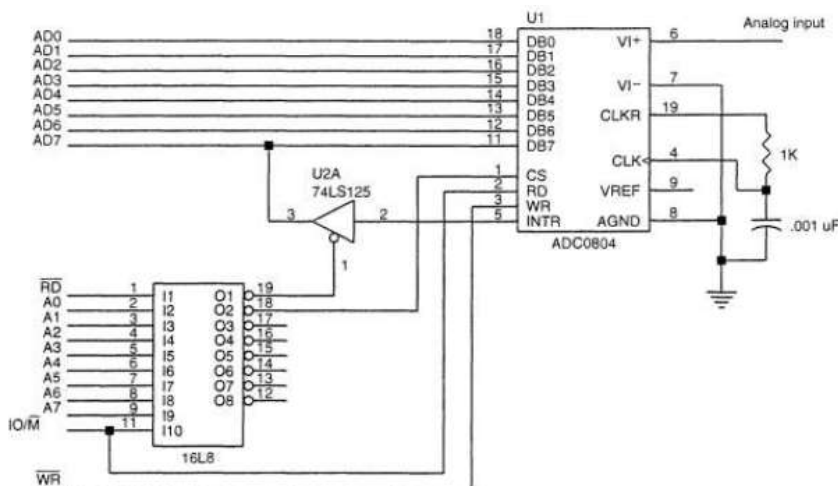
ADC0804

преобразуване с импулс WR, изчаква се INTR да се върне на ниво логическа 0 и след това се четат данните от преобразувателя. Ако се използва времево закъснение повече от 100 μ s време, не е необходимо да се тества INTR линията. Друга възможност е да се свърже краче INTR към вход за прекъсване, така че да се получи прекъсване, когато преобразуването приключи.

Свързване на ADC0804 към микропроцесора.

ADC0804 е свързан с микропроцесора 8086, както е показано на Фигура 10-55. VRFF сигналът не е свързан никъде.

FIGURE 10-55 The ADC0804 interfaced to the microprocessor



Да предположим, че ADC0804 е разположен на 8-битов I/O порт с адрес 40H за данни и порт с адрес 42H за INTR сигнал и е необходима процедура за стартиране и четене на данни от ADC. Тогава бит INTR се проверява чрез периодично четене докато е 1. Когато той стане логическа 0, процедурата завършва. В AL се съдържа преобразувания цифров код.

РЕЗЮМЕ I/O

Микропроцесорите 8086-Pentium Pro имат два основни типа I/O инструкции: IN и OUT. Инструкцията IN въвежда данни от външно I/O устройство в регистъра AL (8-битов) или AX (16-битов). Инструкцията IN е налична като инструкция за фиксиран порт, инструкция за променлив порт или инструкция за низ (80286-Pentium Pro) INSB или INSW. Инструкцията OUT извежда данни от AL или AX към външно I/O устройство и също е налична като фиксирана, променлива или низова инструкция OUTSB или OUTSW. Инструкцията за фиксиран порт използва 8-битов адрес на I/O порт, докато инструкциите за вход/изход с косвена адресация и при низове използват 16-битов номер на порт, зареден в DX регистъра.

Изолираният I/O, понякога наричан директен I/O, използва отделна карта за I/O пространството, освобождавайки цялата памет за използване от програмата. Изолираният I/O използва инструкциите IN и OUT за прехвърляне на данни между I/O устройството и микропроцесора. Структурата за управление на I/O пространството използва IORC (контрол за четене на I/O) и IOWC (контрол на запис на I/O) плюс сигналите за избор на банка BHE и BLE (АО на 8086_и 80286), за да въздейства на I/O трансфера. Ранните микропроцесори 8086/8088 използват M/Ю (IO/M) сигнал с RD и WR за генериране на I/O контролни сигнали. При обединеното с паметта I/O пространство се използва част от адресите на паметта за I/O трансфери. Това намалява количеството налична памет, но отменя необходимостта от използване на сигналите IORC и IOWC за I/O трансфери. В допълнение, всяка инструкция, която адресира клетка в паметта, използвайки който и да е режим на адресиране, може да се използва за прехвърляне на данни между микропроцесора и I/O устройството, използвайки този вид I/O пространство. Входните устройства са буферирани, така че I/O данните са свързани само към шината за данни по време на изпълнението на IN инструкцията. Буферът е или вграден в програмируемо периферно устройство, или е разположен отделно. Всички изходни устройства използват D-тригери за запис на изходни данни по време на изпълнението на инструкцията OUT. Това е необходимо, защото данните се появяват на шината за данни за по-малко от 100 ns за инструкция OUT и повечето изходни устройства изискват данните за по-дълго време.

16-битовата шина за данни, често се нуждае от отделни стробове за запис, горен и долен байт, за операции по запис във порт. По същия начин 80486 и Pen-tium/Pentium Pro също имат I/O подредени в банки. Декодерът на I/O порт много прилича на декодера на адрес на паметта, с изключение на това, че вместо да декодира целия адрес, декодерът на I/O порт декодира само 16-битов адрес за инструкции при косвена адресация и често 8-битов номер на порт за фиксирани I/O инструкции.

82C55 е програмируем периферен интерфейс (PPI), който има 24 I/O извода, които се програмират в две групи от по 12 извода (група А и група В). 82C55 работи в три режима: прост I/O (режим 0), строб I/O (режим 1) и двупосочен I/O (режим 2). Когато 82C55 е свързан с 8086, работещ на 8 MHz, ние вмъкваме два допълнителни шинни състояния на очакване, тъй като скоростта на микропроцесора е по-висока, отколкото 82C55 може да работи.

8254 е програмируем интервален таймер, който съдържа три 16-битови брояча, които отчитат в двоичен или двоично кодиран десетичен знак (BCD). Всеки брояч е независим един от друг и работи в шест различни режима. Шестте режима на брояча са (1) брояч на събития, (2) моностабилен мултивибратор с повторно задействане, (3) импулсен генератор, (4) генератор на квадратни вълни, (5) софтуерно задействан импулсен генератор и (6) хардуерен -задействан импулсен генератор.

16550 е програмируем комуникационен интерфейс, способен да приема и предава асинхронни серийни данни.

DAC0830 е 8-битов цифрово-аналогов преобразувател, който преобразува цифров сигнал в аналогово напрежение в рамките на 1 μ s. ADC0804 е 8-битов аналогово-цифров преобразувател, който преобразува аналогов сигнал в цифров сигнал в рамките на 100 μ s.